

深圳大学实验报告

课程名称 计算机视觉

项目名称 实验一：图像增强应用实践

学 院 数学科学学院

专 业 信息与计算科学（数学与计算机实验班）

指导教师 沈琳琳、 周杰

报 告 人 黄嘉聪 学号 2023192044

实验时间 2024 年 9 月 2 日至 2024 年 10 月 13 日

实验报告提交时间 2024 年 10 月 9 日

教务处制

一、实验目的与要求

实验目的：

1. 熟悉图像表示及基本元素、通道操作；
2. 掌握基本图像增强方法；
3. 掌握 OpenCV 计算机视觉库；

实验要求：

1. 实验提交文件为实验报告和相关程序代码，以压缩包的形式提交，实验报告命名规则为“计算机视觉-学号-姓名-实验报告 1.doc”，其他文件打包成压缩文件，命名为“计算机视觉-学号-姓名-实验报告 1-其他.zip”；

2. 所有素材和参考材料需列明出处，实验报告中的图片和程序代码建议标注个人水印或标识信息：姓名，班级，学号信息；

二、实验内容与方法

实验内容：

(1) 提取给定图像 RGB 和 HSV 颜色空间各通道下的图像；完成给定 RGB、HSV 颜色空间下各通道图像的合并，生成一张彩色图像；提取与合并操作需给出关键核心代码；

(2) 不调用库函数，自己动手编程实现图像增强相关方法，并与 OpenCV 的库函数进行效果对比分析。

三、实验步骤与过程

(1) 提取给定图像 RGB 和 HSV 颜色空间各通道下的图像；完成给定 RGB、HSV 颜色空间下各通道图像的合并，生成一张彩色图像；提取与合并操作需给出关键核心代码；

① 编写相关代码，作好图像处理的输入输出预备。

```
1  import cv2
2  import numpy as np
3  import os
4
5  # 创建输出文件夹
6  output_folder = 'F:\\python\\Computer_Vision\\output_image'
7  if not os.path.exists(output_folder):
8      os.makedirs(output_folder)
9  # 获取“测试图像”文件夹中的所有图像文件
10 folder_path = 'F:\\python\\Computer_Vision\\test_image'
11 image_files = [f for f in os.listdir(folder_path) if os.path.isfile(os.path.join(folder_path, f))]
12
13 # 对每个图像文件进行处理
14 for image_file in image_files:
15     image_path = os.path.join(folder_path, image_file)
16     output_path = os.path.join(output_folder, os.path.splitext(image_file)[0])
```

图一 输入输出预备代码

- ② 调用 OpenCV 的库函数 imread()进行图像的读取；调用 split()进行 RGB 颜色空间各通道图像的提取；调用 imwrite()进行图像的输出；

```
1  import cv2
2  import numpy as np
3  import os
4
5  # 创建输出文件夹
6  output_folder = 'F:\\python\\Computer_Vision\\output_image'
7  if not os.path.exists(output_folder):
8      os.makedirs(output_folder)
9
10 def process_image(image_path, output_path):
11     # 读取图像
12     image = cv2.imread(image_path)
13     # 提取RGB颜色空间各通道图像
14     b_channel, g_channel, r_channel = cv2.split(image)
15     # 保存各通道图像
16     cv2.imwrite(os.path.join(output_path, 'channel_1_R.png'), r_channel)
17     cv2.imwrite(os.path.join(output_path, 'channel_3_B.png'), b_channel)
18     cv2.imwrite(os.path.join(output_path, 'channel_2_G.png'), g_channel)
19 # 获取“测试图像”文件夹中的所有图像文件
20 folder_path = 'F:\\python\\Computer_Vision\\test_image'
21 image_files = [f for f in os.listdir(folder_path) if os.path.isfile(os.path.join(folder_path, f))]
22
23 # 对每个图像文件进行处理
24 for image_file in image_files:
25     image_path = os.path.join(folder_path, image_file)
26     output_path = os.path.join(output_folder, os.path.splitext(image_file)[0])
27     if not os.path.exists(output_path):
28         os.makedirs(output_path)
29     process_image(image_path, output_path)
```

图二 RGB 颜色通道提取代码

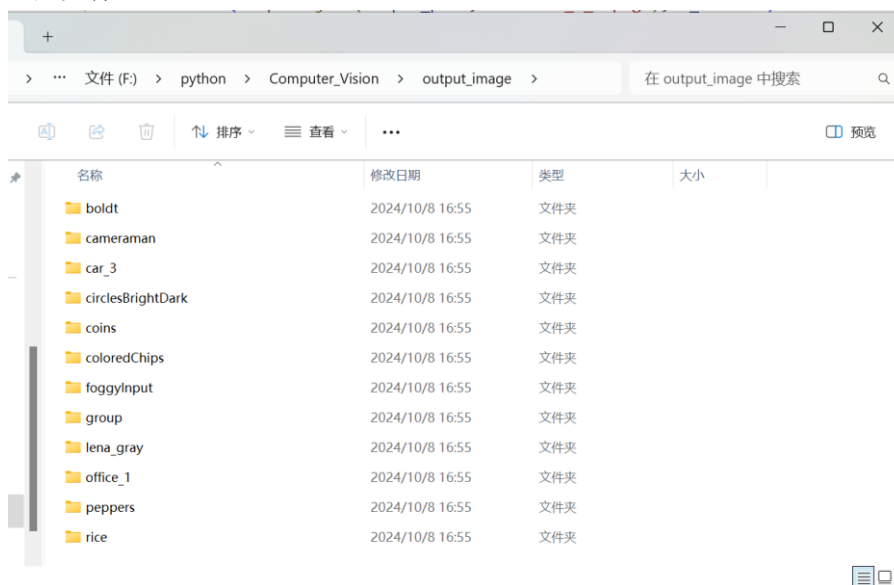
- ③ 调用 OpenCV 的库函数 `cvtColor()` 将原图像转化为 HSV 颜色空间。然后进行②中的操作将 HSV 颜色各通道图像的提取并输出。最后将颜色空间合并并输出。

```
1 import cv2
2 import numpy as np
3 import os
4
5 # 创建输出文件夹
6 output_folder = 'F:\\python\\Computer_Vision\\output_image'
7 if not os.path.exists(output_folder):
8     os.makedirs(output_folder)
9
10 def process_image(image_path, output_path):
11     # 读取图像
12     image = cv2.imread(image_path)
13     # 提取RGB颜色空间各通道图像
14     b_channel, g_channel, r_channel = cv2.split(image)
15     # 保存各通道图像
16     cv2.imwrite(os.path.join(output_path, 'channel_1_R.png'), r_channel)
17     cv2.imwrite(os.path.join(output_path, 'channel_3_B.png'), b_channel)
18     cv2.imwrite(os.path.join(output_path, 'channel_2_G.png'), g_channel)
19     # 转换为HSV颜色空间
20     hsv_image = cv2.cvtColor(image, cv2.COLOR_BGR2HSV)
21     h_channel, s_channel, v_channel = cv2.split(hsv_image)
22     # 保存各通道图像
23     cv2.imwrite(os.path.join(output_path, 'channel_1_H.png'), h_channel)
24     cv2.imwrite(os.path.join(output_path, 'channel_2_S.png'), s_channel)
25     cv2.imwrite(os.path.join(output_path, 'channel_3_V.png'), v_channel)
26 # 获取“测试图像”文件夹中的所有图像文件
27 folder_path = 'F:\\python\\Computer_Vision\\test_image'
28 image_files = [f for f in os.listdir(folder_path) if os.path.isfile(os.path.join(folder_path, f))]
29
30 # 对每个图像文件进行处理
31 for image_file in image_files:
32     image_path = os.path.join(folder_path, image_file)
33     output_path = os.path.join(output_folder, os.path.splitext(image_file)[0])
34     if not os.path.exists(output_path):
35         os.makedirs(output_path)
36     process_image(image_path, output_path)
```

图三 提取与合并通道图像完整代码

- ④ 结果展示：

生成文件：



图四 提取、合并通道图像输出文件夹

例：boldt.jpg 对应生成的图片：



图五 boldt 文件夹

例：boldt 中图片 channel_1_R.png（即 boldt.jpg 分离出的红色通道图片）：



图六 boldt.jpg 分离出的红色通道图片

(2) 不调用库函数，自己动手编程实现图像增强相关方法，并与 OpenCV 的库函数进行效果对比分析。

- ① 图像增强的相关方法包括对比度增强、亮度增强、直方图均衡化。运用对比度增强、亮度增强的公式编写对应的处理函数令函数返回对应处理后的图像。

```
10 # 对比度增强
11 def contrast_enhance(image, alpha):
12     enhanced_image = np.clip(alpha * image, 0, 255).astype(np.uint8)
13     return enhanced_image
```

图七 手动编写的对比度增强函数

```
15 # 亮度增强
16 def brightness_enhance(image, beta):
17     # 确保处理彩色图像时每个通道都进行亮度增强
18     if len(image.shape) == 3: # 彩色图像
19         # 将图像从RGB转换为HSV
20         hsv = np.zeros_like(image, dtype=np.float32)
21         for i in range(image.shape[0]):
22             for j in range(image.shape[1]):
23                 r, g, b = image[i, j] / 255.0
24                 mx = max(r, g, b)
25                 mn = min(r, g, b)
26                 diff = mx - mn
27                 if mx == mn:
28                     h = 0
29                 elif mx == r:
30                     h = (60 * ((g - b) / diff) + 360) % 360
31                 elif mx == g:
32                     h = (60 * ((b - r) / diff) + 120) % 360
33                 elif mx == b:
34                     h = (60 * ((r - g) / diff) + 240) % 360
35                 if mx == 0:
36                     s = 0
37                 else:
38                     s = diff / mx
39                 v = mx
40                 hsv[i, j] = [h, s, v]
41
42     # 增强亮度
43     hsv[:, :, 2] = np.clip(hsv[:, :, 2] * 255 + beta, 0, 255) / 255.0
44
45     # 将图像从HSV转换回RGB
46     enhanced_image = np.zeros_like(image, dtype=np.uint8)
47     for i in range(hsv.shape[0]):
48         for j in range(hsv.shape[1]):
49             h, s, v = hsv[i, j]
50             c = v * s
51             x = c * (1 - abs((h / 60) % 2 - 1))
52             m = v - c
53             if 0 <= h < 60:
54                 r, g, b = c, x, 0
55             elif 60 <= h < 120:
56                 r, g, b = x, c, 0
57             elif 120 <= h < 180:
58                 r, g, b = 0, c, x
59             elif 180 <= h < 240:
60                 r, g, b = 0, x, c
61             elif 240 <= h < 300:
62                 r, g, b = x, 0, c
63             elif 300 <= h < 360:
64                 r, g, b = c, 0, x
65             enhanced_image[i, j] = [(r + m) * 255, (g + m) * 255, (b + m) * 255]
66     else: # 如果没彩色通道就是灰度图像
67         enhanced_image = np.clip(image + beta, 0, 255).astype(np.uint8)
68     return enhanced_image
```

图八 手动编写的亮度增强函数代码

- ② 在直方图均衡化的处理函数中编写计算直方图、计算累积分布函数、归一化、最后使用累积分布函数进行直方图均衡化后令函数返回对应处理后的图像。

```
71 def equalize_hist(image):
72     # 计算直方图
73     hist, bins = np.histogram(image.flatten(), 256, [0, 256])
74     # 计算累积分布函数
75     cdf = hist.cumsum()
76     # 归一化
77     cdf_normalized = cdf * (255 / cdf[-1])
78     # 使用累积分布函数进行直方图均衡化
79     equalized_image = np.interp(image.flatten(), bins[:-1], cdf_normalized)
80     return equalized_image.reshape(image.shape).astype(np.uint8)
```

图九 手动编写的直方图均衡化函数代码

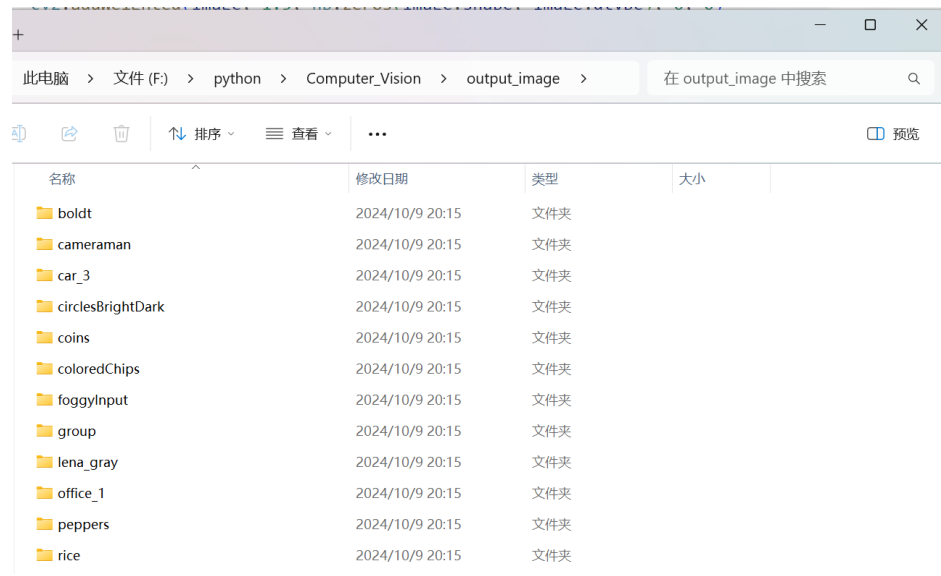
- ③ 在对输入图像处理的主函数中，使用上述编写的函数对读取的图像进行处理后输出。然后在主函数中再次使用 OpenCV 自带的库对图像进行处理后输出

```
81 def process_image(image_path, output_path):
82     # 读取图像
83     image = cv2.imread(image_path)
84
85     # 自行进行图像增强操作并输出
86     # 对比度增强
87     contrast_image = contrast_enhance(image, 1.5) # alpha > 1 增强对比度
88     cv2.imwrite(os.path.join(output_path, 'contrast_image.png'), contrast_image)
89     # 亮度增强
90     brightness_image = brightness_enhance(image, 50) # beta > 0 增强亮度
91     cv2.imwrite(os.path.join(output_path, 'brightness_image.png'), brightness_image)
92     # 直方图均衡化
93     gray_image = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
94     equalized_image = equalize_hist(gray_image)
95     cv2.imwrite(os.path.join(output_path, 'equalized_image.png'), equalized_image)
96
97     # 使用OpenCV库函数进行图像增强操作并输出以对比
98     # 对比度增强
99     contrast_image_cv = cv2.addWeighted(image, 1.5, np.zeros(image.shape, image.dtype), 0, 0)
100    cv2.imwrite(os.path.join(output_path, 'contrast_image_cv.png'), contrast_image_cv)
101    # 亮度增强
102    brightness_image_cv = cv2.add(image, 50)
103    cv2.imwrite(os.path.join(output_path, 'brightness_image_cv.png'), brightness_image_cv)
104    # 直方图均衡化
105    equalized_image_cv = cv2.equalizeHist(gray_image)
106    cv2.imwrite(os.path.join(output_path, 'equalized_image_cv.png'), equalized_image_cv)
107
108    # 获取“测试图像”文件夹中的所有图像文件
109    folder_path = 'F:\\python\\Computer_Vision\\test_image'
110    image_files = [f for f in os.listdir(folder_path) if os.path.isfile(os.path.join(folder_path, f))]
111
112    # 对每个图像文件进行处理
113    for image_file in image_files:
114        image_path = os.path.join(folder_path, image_file)
115        output_path = os.path.join(output_folder, os.path.splitext(image_file)[0])
116        if not os.path.exists(output_path):
117            os.makedirs(output_path)
118        process_image(image_path, output_path)
```

图十 处理图像的主函数代码

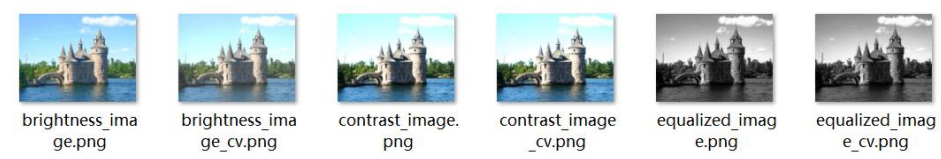
④ 输出结果：

生成文件：



图十一 手动编写函数、OpenCV 库函数处理后图像输出文件夹

例：boldt 生成图片：



图十二 boldt 文件夹

例：boldt 中图片 `brightness_image`（手动编写亮度增强函数处理后图像）：



图十三 手动编写亮度增强函数处理后图像

例：boldt 中图片 `brightness_image_cv`（OpenCV 亮度增强库函数处理后图像）：



图十四 OpenCV 亮度增强库函数处理后图像

四、实验结论或体会

实验结论：

通过（1）中的实验，成功实现了从 RGB 和 HSV 颜色空间中提取图像的各个通道，并分别完成了这些通道的合并操作。具体来说，我们首先使用了 OpenCV 库中的函数将 RGB 颜色空间中的各个通道图并保存了这些图像，然后使用 OpenCV 库中的函数将图像从 RGB 颜色空间转换到 HSV 颜色空间，同理再次使用 OpenCV 库中的函数提取 HSV 颜色空间中的各个通道图像并保存了这些图像。接着，我们将这些通道合并成原始的彩色图像，并保存了这些图像。最后，我们通过比较保存下来的 RGB 和 HSV 颜色空间下的图像，观察了不同颜色空间对图像处理的影响。

通过（2）中的实验，我们成功手动实现了图像增强的相关方法，并与 OpenCV 库函数进行了效果对比分析。具体来说，我们首先实现了图像的对比度增强和亮度增强的函数，然后实现了图像的直方图均衡化的函数。接着，我们使用 OpenCV 库函数实现了相同的图像增强方法，最后通过主函数中调用图像增强的函数进行图像的输出，最后通过输出的图像比较了两种方法的处理效果。

实验体会：

在（1）的实验内容当中，我学习到了在 Python 中 OpenCV 库的一些主要函数。也体会到了 OpenCV 库对于图像处理的方便快捷，更为深刻的图像处理中提取通道和合并通道的作用：通过提取通道，可以更深入地理解图像的构成，而通过合并通道，可以由此恢复或生成新的图像。通过（1）中的实验，对于图像处理的基本原来和应用有了更深刻的理解。

除此之外，使用了 Python 中的 os 库函数进行了大范围读取图像和存储图像的操作。在使用 os 库的过程当中出现了相对地址失效的问题，最后通过将地址改为绝对地址解决。对于 Python 中的读取操作等有了更深刻的认识。

在（2）的实验内容当中，通过自己手写的函数更为深刻的理解了图像增强的方法当中对于图像的处理方法。以此为契机也更深刻的认识到了 OpenCV 中库函数的运行原理，同时也更为深刻的理解了图像增强的各个方法对于图像整体的作用。

在技术上，在亮度增强的函数写作当中，一开始由于是直接对图像的亮度部分进行增强，导致输出出来的灰白图像虽然没问题，但是导致了彩色图像的严重的颜色偏移问题。最后通过将两种类型的图像分开，再将彩色图像转化为 HSV 图像后，通过对于各个颜色通道进行单独的亮度处理，解决了最后的颜色偏移。

在直方图均衡化的函数写作当中，一开始强行将均衡化后的累计分布函数直接套用到直方图当中，导致最终输出的图像当中出现了大片的严重损毁图像的马赛克。最后通过归一化的操作将该问题进行修复。

在设计上，将手写的各个方法进行函数的封装，使得后面在遍历各大图像时可以在主函数中直接调用函数，提高了整个代码的对称性、封装性和可读性。使得代码运行更为合理流畅。

五、思考题

对比不同图像增强方法的优势、劣势，包括其合适的应用场景。

① 直方图均衡化：

优势：能够自动调整图像的灰度级分布，使图像更加清晰。

劣势：可能导致图像的某些细节丢失或对比度不自然的过分增强，其次如果对处理的数据不加选择，可能会增加图像背景噪声的对比度从而影响图像。

应用场景：适用于背景和前景都太亮或者太暗的图像，以及对比度较低、灰度级分布不均匀的图像。

② 对比度增强：

优势：简单易实现，能够显著增强图像的对比度。

劣势：可能导致图像的颜色失真或噪声增加，其次是对于对比度已经很高的图像，增强效果可能不明显。

应用场景：对比度增强适用于灰度值分布比较均匀的图像。也可以用于改善低光条件下的图像质量。

③ 亮度增强：

优势：能够直接调整图像的明亮程度，使图像更加清晰。

劣势：过度增强可能导致图像过曝，细节丢失，其次是如果只增强亮度而同时不考虑对于对比度的影响，可能导致最终处理后的图像看起来不自然。

应用场景：适用于光线不足或需要提高图像亮度的场景，如夜景照片等，也可以用于改善低亮度图像整体的视觉效果，使图像更加明亮、清晰。

总的来说，对于各种图像，不同的方法效果会不一样，使用合适的方法才能达到最好的效果。

指导教师批阅意见:

成绩评定：

指导教师签字：沈琳琳、周杰

2024 年 10 月 13 日

备注:

注：1、报告内的项目或内容设置，可根据实际情况加以调整和补充。